DHBW Stuttgart

Karsten Keßler

Datenbanksysteme
13. Teil – Interne Strukturen

Kurs Informatik 24ITA

DHBW Stuttgart

Inhalte der heutigen Vorlesung

- Einordnung in den Vorlesungsverlauf
- Überlegungen zu Indexstrukturen
- ISAM
- B-Bäume
- Bitmap-Indexstrukturen

Übersicht der Gesamtvorlesung

ANSI SPARC Architektur – Interne Ebene

Drei-Ebenen-Architektur einer Datenbank: wie man Datenbankzugriffe in externe, konzeptuelle und interne Sicht trennt, um mehr Flexibilität und Datenunabhängigkeit zu erreichen.

Wenn man die interne Ebene ändert (z. B. Index hinzufügen), braucht man nichts an der konzeptuellen oder externen Ebene zu ändern (physische Datenunabhängigkeit).
Wenn man das konzeptuelle Schema erweitert, müssen die externen Sichten teils nicht angepasst werden

Thema	Inhalte (Auszug)
1 Einführung	Einführung, Begriffe, Eigenschaften Funktionen, Datenintegration
2 Datenbankmodelle	Architektur und SPARC, Verschiedene Datenbankmodelle, Entwicklung
3 ERM	Entity-Relationship-Modell, konzeptueller Entwurf, Entwurfsprozess
4 Relationales Modell	Relationales Datenbankmodell im Detail
5 Relationale Algebra	Operatoren, Anwendung, Nützlichkeit
6 Normalformen	Normalisierung von Datenbanken, funktionale Abhängigkeiten, Synthesealgorithmus
7 Vektordatenbanken	Einstieg in Vektordatenbanken
8 SQL DDL	Datendefinition mit SQL
9 SQL DML	Datenmanipulation mit SQL
10 Transaktionen	ACID, Grundlegende Schwierigkeiten bei Mehrbenutzersystemen, gängige Lösungen
11 Integrität	Integrität im relationalen Modell wiederholt, Umsetzung (in SQL)
12 Sichten	Sichten (in SQL), Vor- und Nachteile, Umgang mit Sichten
13 Indizes	Interne Strukturen, Nützlichkeit für Anfragen
14 Optimierung	Optimierung von Datenbanken
15 SQL Server	Administration des SQL Server
16 Handout	Handout: Datenbanksysteme – Zusammenfassung

Dateiorganisationsformen

→ Dateiorganisationsformen beschreiben, wie Datensätze physisch in Dateien oder Blöcken abgelegt werden. Hier sind die wichtigsten:

- **Heap-Organisation**

Tupel werden unsortiert in Dateien / Blöcken abgelegt In einer Heap-Format-Dateiorganisation existiert in der Regel keine bestimmte Reihenfolge oder Struktur für die Speicherung von Daten in der Datei. Es gibt keine vordefinierte Hierarchie oder Reihenfolge, in der die neuen Daten gespeichert werden müssen. Sie werden lediglich am Ende der Datei angehängt. Dies führt zu einer unkomplizierten und zügigen Einfügeoperation, da keine Neuordnung der Daten notwendig ist.

- **Sequentielle Organisation**

Die Tupel werden in einer festgelegten Reihenfolge sortiert gespeichert.
Bei der sequentiellen Dateiorganisation handelt es sich um eine Vorgehensweise zur Speicherung von Daten in einer Datei in Folge oder nacheinander. Die Datei speichert jeden Datensatz in der Reihenfolge, in der er eingefügt wird: Datensätze werden sortiert nach einem Schlüssel gespeichert.
Diese Art der Organisation ist unkompliziert und wirksam, vor allem für Anwendungen, die vor allem sequenzielle Zugriffe benötigen, wie zum Beispiel das Lesen oder Schreiben von Daten in der Reihenfolge ihres Auftretens.
Sequentielle Organisation = Grundlage für Binärsuche.

- **Hash-Organisation**

Die Tupel werden „gestreut“ gespeichert.
Die Hash-Dateiorganisation ist ein Verfahren, mit dem Daten in einer Datei gespeichert und verwaltet werden, wobei eine Hashfunktion genutzt wird. Dabei erfolgt eine Strukturierung der Daten in Form einer Tabelle. Mithilfe der Hashfunktion wird der Schlüssel jedes Datensatzes in einen Index umgewandelt, der die Position des Datensatzes in der Datei festlegt. Diese Methode der Organisation wird oft genutzt, um einen unmittelbaren und zügigen Zugang zu Daten zu ermöglichen.

- **Mehrdimensionale Organisation**

Ausschlaggebend ist nicht nur ein, sondern mehrere

Schlüssel Bei der mehrdimensionalen Dateiorganisation handelt es sich
um eine Methode zur Organisation von Daten in einer Datei in verschiedenen
Dimensionen. Die mehrdimensionale Organisation erlaubt im Unterschied
zu herkömmlichen Organisationsmethoden, die häufig eine hierarchische oder
lineare Struktur benutzen, den Zugang zu Daten über unterschiedliche
Dimensionen oder Achsen.

Verfahren	Beschreibung/Vorteil	Nachteil
Heap-Organisation	Unsortierte Ablage, sehr schnelle Inserts, geringer Verwaltungsaufwand	Langsame Suche, keine Ordnung, evtl. vollständige Dateidurchsuchung erforderlich
Sequentielle Organisation	Sortierte Speicherung, schnelle Suche durch Binärsuche, effiziente Range-Queries	Einfügen kann aufwändig sein, da Daten umsortiert werden müssen
Hash-Organisation	Direkter, schneller Zugriff über Hashfunktion bei exakten Schlüsselabfragen, oft nur ein Dateizugriff nötig	Nicht geeignet für Bereichsanfragen, mögliche Hash-Kollisionen können Performance-Einbußen verursachen
Mehrdimensionale Organisation	Optimiert für Suchen über mehrere Attribute (z. B. geografische Daten) und komplexe, mehrdimensionale Abfragen	Komplexere Implementierung und Verwaltung, höherer Ressourcenbedarf

Welche Speicherstrukturen sind möglich?

→ Mögliche Speicherstrukturen in Datenbanksystemen werden meist in Form von Index-Methoden eingesetzt, um Datenzugriffe (Suchen, Einfügen, Löschen) zu beschleunigen. Hier eine kurze Beschreibung der wichtigsten Verfahren:

- ISAM (Indexed Sequentiell Access Method), ähnlich Telefonbuch

In der Informatik wird ISAM (Indexed Sequential Access Method) als Speicherstruktur genutzt, um Daten in einer Datei zu organisieren und zu verwalten. Es vereint sequentielle und indexierte Zugriffsmethoden, um einen effektiven Zugang zu den Daten zu gewährleisten. ISAM wurde konzipiert, um mithilfe von Indizes die Wirksamkeit von sequenziellen Dateiorganisationen zu steigern.

- B-Bäume – der „Klassiker“ im Bereich der Datenbanken

B-Bäume werden in der Informatik oft genutzt, um Daten in einer Datei oder einem Datenbanksystem zu organisieren und zu managen. Sie sind so konzipiert, dass sie Daten effizient suchen, einfügen, löschen und aktualisieren können, vor allem in umfangreichen Datensätzen. B-Bäume sind darauf ausgerichtet, Daten auf Festplatten oder anderen sekundären Speichermedien effizient abzurufen.

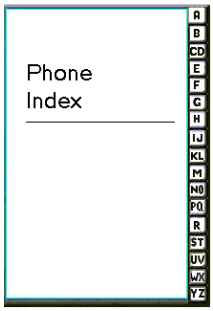
- Hashverfahren – spezialisiert auf Gleichheitsabfragen

Ein Hashverfahren beschreibt die Anwendung von Hashfunktionen zur effizienten Organisation und Abruf von Daten in einem Speichermedium. Die Organisation von Daten in einer Datei erfolgt durch die Verwendung von Hashverfahren, die auf ihren Hashwerten basieren. In Datenbanken kann dies beispielsweise genutzt werden, um den Zugang zu Datensätzen zu erleichtern. Es ist möglich, Datensätze mithilfe von Hashfunktionen direkt nach ihrem Schlüssel zu lokalisieren, ohne dass die Datei sequenziell durchsucht werden muss.

- Bitmap Indexstrukturen – bei Kombination von Attributen mit „wenigen“ Werten

Bitmap-Indexstrukturen können Indexdaten effektiv in Datenbanken und anderen Speicherlösungen speichern. Sie dienen dazu, dass in Spalten mit einem begrenzten Wertebereich Daten schnell abgerufen werden können, vor allem bei Datensätzen mit zahlreichen Wiederholungen und einer begrenzten Anzahl eindeutiger Werte. Bitmap-Indexstrukturen ermöglichen einen effizienten und platzsparenden Zugang zu Daten in Spalten mit einem begrenzten Wertbereich. Bei Data-Warehousing-Anwendungen und analytischen Abfragen sind sie besonders hilfreich, da dort schnelle Abfragen über große Datensätze notwendig sind.

ISAM – Index-Sequential Access Method



Das Telefonbuch veranschaulicht, wie durch eine geordnete (sortierte) Datenstruktur und einem Index, die schnelle Auffindbarkeit von Einträgen gewährleistet wird (wie bei ISAM in Datenbanksystem).
→ ISAM ist statisch – Inserts führen zu Overflow-Seiten

„Daumenindex“ Geordnetes Speichern der Datensätze und Schlüssel - Index ist statisch aufgebaut und wird sequentiell durchsucht

⇨

Index:
ID: 001, Pointer: Location of Alice's record
ID: 002, Pointer: Location of Bob's record
ID: 003, Pointer: Location of Charlie's record

Daten:
ID: 001, Name: Alice, Position: Developer
ID: 002, Name: Bob, Position: Designer
ID: 003, Name: Charlie, Position: Manager

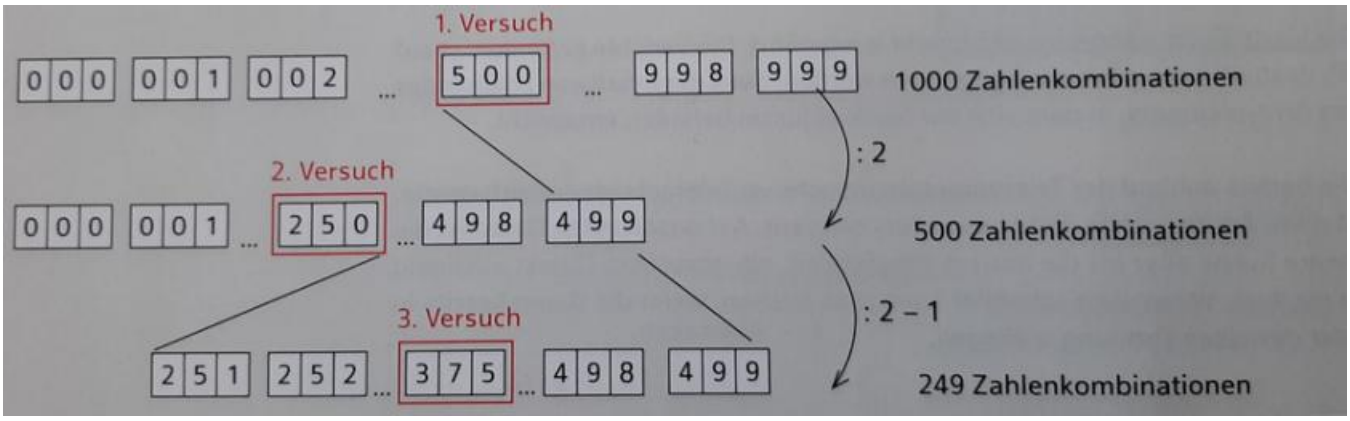
→ Über Index gelangt man schneller zu den Daten, als über alle Daten zu lesen.

Suchen: Binärsuche im Index - lineares Durchsuchen der Seite und ihrer Nachfolgeseiten bis zu einem „zu großen“ Schlüssel

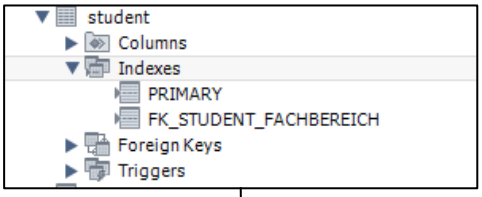
Einfügen: Aufwändig! Bei voller Datenseite Ausgleich mit Nachbarseite versuchen, ansonsten neue Seite anlegen und Index verschieben

Binäre Suche in sortierten Daten

Die binäre Suche halbiert in jedem Schritt den Suchbereich. Das macht sie sehr effizient: Bei 1000 Elementen sind im schlimmsten Fall nur etwa 10 Vergleiche nötig (weil $\log_2(1000) \approx 10$).
Voraussetzung: Die Daten müssen sortiert sein!
Binäre Suche braucht höchstens $\log_2(n)$ Schritte bei n Elementen.



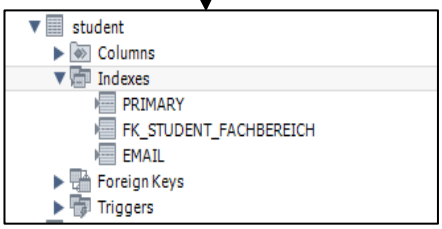
Indexformen (1)



- Gewöhnliche (nicht spezialisierte) Indexarten
- Einzelindizes:

  Eine Indexstruktur in einer Datenbank, die auf einer einzigen Spalte oder einem einzigen Attribut einer Tabelle basiert, wird als Einzelindex bezeichnet. Mit einem Einzelindex können Datensätze, die auf den Werten in dieser bestimmten Spalte basieren, schnell abgerufen werden.

  alter table student add index(EMAIL)
  DROP INDEX EMAIL ON student;



Der Index besteht aus einer geordneten Liste aller Werte der Spalte 'EMAIL'. Durchsucht der User die Datenbank, durchsucht das DBMS den Index und lädt die Speicherseite, in der der Datensatz gespeichert ist.

- Mehrspaltige Indizes:

  CREATE INDEX index_2 ON student (email, vorname);

Eine Indexstruktur in einer Datenbank ist ein mehrspaltiger Index, der auch als zusammengesetzter Index oder verbundener Index bezeichnet wird. Diese basiert auf mehreren Spalten oder Attributen einer Tabelle. Ein mehrspaltiger Index erlaubt im Unterschied zum Einzelindex, der ausschließlich auf einer Spalte basiert, einen zügigen Zugang zu Datensätzen, die auf Wertkombinationen in diesen Spalten basieren.

Einzel:
*SELECT * FROM student WHERE email = 'max@example.com';*
*SELECT * FROM student ORDER BY nachname;*

Mehrspaltig:
*SELECT * FROM student WHERE email = 'a@b.cd' AND vorname = 'Anna';*

- Unique-Indizes

  create unique index idx_matnr on student (matnr)

  Ist eine Variante zum gewöhnlichen Index: Eine Indexstruktur in einer Datenbank, die sicherstellen soll, dass die Werte in der indexierten Spalte oder Spaltenkombination eindeutig sind, wird als Unique-Index bezeichnet. In anderen Worten lässt ein Unique-Index in den indexierten Spalten keine Doppelgänger zu.

- Primärindex

  In einer Datenbank ist ein Primärindex, der auch als Primärschlüsselindex oder Primary Key Index bekannt ist, eine besondere Art von Index. Zur Gewährleistung einer eindeutigen Identifizierung von Datensätzen wird er auf einer oder mehreren Spalten einer Tabelle erstellt. Normalerweise erfolgt die Erstellung des Primärindex auf der Primärschlüsselspalte(n) der Tabelle, die für jeden Datensatz in der Tabelle eine eindeutige Kennung darstellen. Eine Tabelle hat genau einen Primärschlüssel (ggf. zusammengesetzt aus mehreren Spalten). Das DBMS erzeugt bei CREATE TABLE … PRIMARY KEY(..) einen Primärindex.

Primärindex (nur 1x erlaubt, keine NULLs) und Unique-Index (mehrere erlaubt, aber keine Duplikate):

CREATE TABLE student (matrnr INT PRIMARY KEY, email VARCHAR(100) UNIQUE, vorname VARCHAR(50));

- Index für Fremdschlüssel

FK-Spalten müssen nicht zwingend mit eine Index ausgestattet werden. Es hängt von den Abfragen ab, die sie oft ausführen. MySQL legt für Fremdschlüssel automatisch einen Index an, falls keiner existiert.
Ein Index für Fremdschlüssel, der auch unter dem Namen Fremdschlüsselindex oder Foreign Key Index bekannt ist, ist ein Index in einer Datenbank. Dieser wird auf die Spalte oder Spalten einer Tabelle angewendet, die als Fremdschlüssel dienen. Eine Spalte oder Spaltengruppe in einer Tabelle, die auf die Primärschlüsselspalte(n) einer anderen Tabelle verweist, wird als Fremdschlüssel bezeichnet. Zur Optimierung des Zugriffs auf die Daten, die mit diesen Fremdschlüsselbeziehungen verknüpft sind, wird ein Fremdschlüsselindex erstellt.

Indexes in Table

Key	Type	Unique	Columns
PRIMARY	BTREE	YES	MATNR
FK_STUDENT_FAC...	BTREE	NO	FB
EMAIL	BTREE	NO	EMAIL

Eltern:
CREATE TABLE fachbereich (id INT PRIMARY KEY, name VARCHAR(100));

Kinder:
CREATE TABLE student (matrnr INT PRIMARY KEY, name VARCHAR(100), fbid INT, FOREIGN KEY (fbid) REFERENCES fachbereich(id) ON DELETE CASCADE);

→ bei vielen Kindernm müssen alle gefunden werden mit fbid = 3.

Mit Index: schnell
Ohne. Full table scan → langsam

Nachteile von Indizes:

Indizes beschleunigen Lesevorgänge, aber verlangsamen Schreibvorgänge und erhöhen den Platzbedarf der Datenbank. Daher sollten außer dem Primärschlüsselindex nur jene Spalten indiziert werden, die häufig gesucht oder sortiert werden.

Speicherplatzbedarf: Indizes können aufgrund der Notwendigkeit zusätzlicher Datenstrukturen, um die Indexinformationen zu speichern, zusätzlichen Speicherplatz in der Datenbank nutzen. Dies könnte vor allem in großen Tabellen mit zahlreichen Indizes zu einem erheblichen Bedarf an Speicher führen.

Aktualiserungskosten: Es kann mehr Zeit in Anspruch nehmen, Daten in einer indizierten Spalte zu aktualisieren als in einer nicht indizierten. Dies ist darauf zurückzuführen, dass die Indexwerte, die damit verbunden sind, auch aktualisiert werden müssen, wenn ein Datensatz aktualisiert wird. Dies kann zu einer Verringerung der Leistung führen, vor allem bei regelmäßigen Updates oder großen Tabellen.

Einschränkung bei Schreibvorgängen: Indizes können die Funktionalität von Schreibvorgängen wie Einfügen, Aktualisieren und Löschen von Daten stören. Indizes können die Geschwindigkeit dieser Vorgänge verlangsamen, besonders wenn viele Indizes vorhanden sind, da bei diesen Operationen auch die entsprechenden Indexwerte aktualisiert werden müssen.

Fragmentierung: Indizes können sich mit der Zeit fragmentieren, vor allem wenn Daten oft hinzugefügt, aktualisiert oder gelöscht werden. Fragmentierte Indizes können sich negativ auf die Leistung der Abfrage auswirken und erfordern regelmäßige Wartung, um sie zu verbessern.

Indexart	Zweck	Vorteil	Nachteil
Einzelindex	Auf eine einzelne Spalte, um Suchzugriffe zu beschleunigen	✅ Schneller Zugriff bei Abfragen nach genau dieser Spalte ✅ Einfache Erstellung	❌ Nur nützlich für diese eine Spalte ❌ Keine Unterstützung bei Kombi-Abfragen
Mehrspaltiger Index	Auf mehrere Spalten, z. B. für kombinierte Suchen und Sortierungen	✅ Sehr effizient bei Kombinationen ✅ Spart mehrere Einzelindizes	❌ Muss exakt in Spaltenreihenfolge verwendet werden ❌ Komplexer in der Planung
Unique-Index	Sichert Eindeutigkeit in Spalten (z. B. E-Mail, Benutzername)	✅ Verhindert doppelte Einträge ✅ Kann auch als Suchindex dienen	❌ Nur sinnvoll bei Spalten mit eindeutigen Werten ❌ Meist nur ein NULL erlaubt (je nach DBMS)
Primärindex	Automatischer Index auf Primärschlüssel-Spalte(n)	Garantiert Eindeutigkeit ✅ Wird automatisch gepflegt ✅ Optimiert JOINs auf PK	❌ Nur einer pro Tabelle erlaubt ❌ Kann nicht NULL sein
Fremdschlüsselindex	Index auf Fremdschlüssel, zur Optimierung von JOINs und referenziellen Aktionen	✅ Schnelle JOINs auf Eltern-Tabellen ✅ Wichtig für ON DELETE CASCADE, SET NULL, etc.	❌ Nicht in allen DBMS automatisch angelegt ❌ Muss manuell gepflegt werden bei Performancebedarf

Speicherverbrauch von Tabellen und Indizes in deiner MySQL-Datenbank, dargestellt mit den Spalten (Sichten als virtuelle Tabellen und daher Speicher NULL)

SELECT table_name, ROUND(SUM(data_length) / 1024 / 1024, 2) AS TableSpaceMB, ROUND(SUM(index_length) / 1024 / 1024, 2) AS IndexSpaceMB FROM information_schema.tablesWHERE table_schema = 'index_demo' GROUP BY table_name;

TABLE_NAME	TableSpaceMB	IndexSpaceMB
adminsicht	NULL	NULL
benutzer	0.02	0.02
benutzerohnerollesicht	NULL	NULL
bestestudentsicht	NULL	NULL
dauinfosicht	NULL	NULL
dozent	0.02	0.02
dozentinfosicht	NULL	NULL
dozentsicht	NULL	NULL
fachbereich	0.02	0.00
gutenotensicht	NULL	NULL
kurs	0.02	0.02
kursdurchschnittsicht	NULL	NULL
kursstatistiksicht	NULL	NULL
log	0.02	0.00
pruefung	0.02	0.03
student	0.02	0.08
studenteninfosicht	NULL	NULL
studentsicht	NULL	NULL
studiengang	0.02	0.00
topstudentsicht	NULL	NULL

Das DBMS bietet im Ausführungsplan fehlende Indizes an.

*EXPLAIN SELECT * FROM student WHERE email = 'august.algebra@example.com'*

Ohne Index:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	student	NULL	ALL	NULL	NULL	NULL	NULL	2	50.00	Using where

Mit Index:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	student	NULL	ref	idx_email	idx_email	403	const	1	100.00	NULL

In der Vorlesung betrachten wir den klassischen B-Baum als Konzept. In der Praxis verwenden DBMS meist B^+-Bäume.

B-Bäume (1)

⇨ Im Hauptspeicher können Binärbäume als Speicherstruktur verwendet werden - diese sind jedoch nicht effizient auf die Seitenstruktur des Hintergrundspeichers abbildbar.

⇨ B-Bäume (≠ Binärbäume) begrenzen die Zahl der Seitenzugriffe.
Ein Binärbaum hat max. 2 Kinder pro Knoten, während ein B-Baum mehrere haben kann
→ besser für plattenbasierte Systeme (Weniger Seitenzugriffe = schneller!).

Definition: B-Baum vom Grad k

Jeder Knoten hat mindestens k und höchstens 2k Schlüsseleinträge (ausser Wurzel)

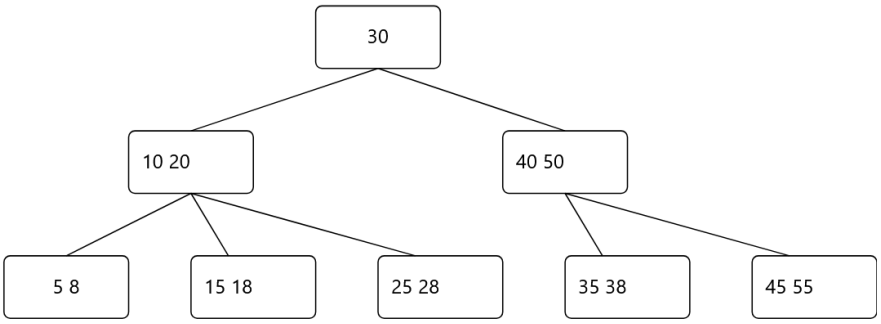
Der Wurzelknoten hat zwischen 1 und k Schlüsseleinträge.

Die Schlüsseleinträge in den Knoten sind **sortiert**.

Jeder Knoten außer den Blattknoten (der "unterste"), der s Schlüsseleinträge besitzt, hat s+1 Kindknoten

Seien S1 ... Sn die Schlüsseleinträge eines Knotens und T0, ..., Tn die Verweise auf Kindknoten, dann gilt:

T0 verweist auf Kindknoten, dessen Schlüsseleinträge kleiner als S1 sind
Ti verweist auf Kindknoten, dessen Schlüsseleinträge >= Si und kleiner al Si+1 sind
Blattknoten besitzen **keine** weiteren Verweise



B-Baum vom Grad k=2

→ Jeder Knoten hat 2-4 Schlüssel (außer Wurzel)
→ Blattknoten auf gleicher Ebene

Ein B-Baum ist eine Baumstruktur zur Speicherung von Indizes durch das DBMS. Jedem Knoten des Baums ist eine ganze Speicherseite (Page) zugeordnet.
Pro Knoten kann ein B-Baum viele Indexeinträge speichern, meist mehrere hundert (verschiedene).*
Sobald ein Knoten nicht mehr für alle Indexeinträge ausreicht, werden die Einträge über mehrere Knoten verteilt. Jeder Indexeintrag gilt dabei als Trennpunkt: der linke Subknoten erhält kleinere der rechte größere Indexeinträge. Wegen der großen Elementanzahl pro Knoten, haben B-Bäume nur eine geringe Tiefe (meistens 3-5), dafür sind sie sehr breit.
Die Ordnungszahl m (!= Grad) gibt an, wie viele Verweise auf Subknoten ein B-Baum-Knoten maximal haben kann.

*) Seitengröße (Page): 16.384 Bytes (16 KB)
Größe eines Indexeintrags: ca. 50 Bytes
Max. Einträge pro Knoten: ≈ 300

Folgende Regeln müssen eingehalten werden:

- Jeder Knoten mit Ausnahme des Root-Knoten muss mindestens zu 50% gefüllt sein.
- Die Anzahl der Links auf Subknoten ist immer um eins größer als die Anzahl der Indexeinträge, ausgenommen sind die Endknoten (Leaf, Blatt), die auf keine weiteren Knoten verweisen.
- Wenn beim Einfügen und Entfernen von Indexeinträgen die max. Anzahl von Elementen überschritten oder minimale unterschritten wird, muss der Baum reorganisiert werden.

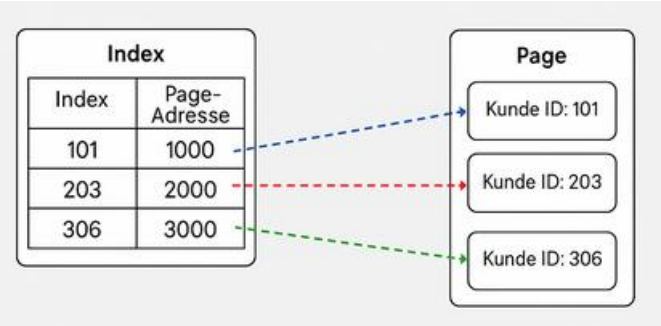
Speicherblöcke (Pages)

Wenn Datensätze geschrieben werden, müssen diese physikalisch auf der Platte (SSD) gespeichert werden. Auch Änderungen des Datensatzes müssen später gespeichert werden. Falls die Änderung mehr Platz benötigt (z.B. ein Bild einfügen), muss möglicherweise ein neuer Speicherort lokalisiert werden.
Die meisten DBMS organisieren die Datenspeicher in Blöcken, die man als Pages bezeichnet. Die Blockgröße variiert je nach DBMS zwischen 4k, 8k und 16k.

Die Rolle des Index

Damit das DBMS beim Zugriff auf einen bestimmten Datensatz nicht immer alle Pages mit allen Datensätzen durchsuchen muss, benötigt es ein Inhaltsverzeichnis. Das sagt aus, an welchem Ort welcher Datensatz abgespeichert ist. Dieses Inhaltsverzeichnis wird als Index bezeichnet.

Ein Index ermöglicht den schnellen Zugriff auf die Datenblöcke (Pages) in einem DBMS.



Vereinfachte Darstellung von Daten in Blöcken (Pages)

Speicherseite mit books-Datensätzen

bookNo = 1; title = An Introduction to Database Systems; edition = 3; publNo = 1; year = 1981;; bookNo = 2; title = The Relational Model for Database Management; edition = 1; publNo = 1; year = 1990;; bookNo = 3; title = Time and Relational Theory: Temporal Databases in the Relational Model and SQL; edition = 2; publNo = 2; year = 2014;; bookNo = 4; title = Database Design and Relational Theory; edition = 2; publNo = 3; year = 2019;;

Speicherseite mit publishers-Datensätzen

publNo = 1; publisher = Addison-Wesley; publCity = Reading;; publNo = 2; publisher = Morgan-Kaufmann; publCity = Burlington;; publNo = 3; publisher = Apress; publCity = New York;;

```
SELECT b.title, p.publisher
FROM books b
JOIN publishers p ON b.pubNo = p.pubNo
WHERE b.year > 2000;
```

WHERE b.year > 2000
→ scannt books-Page (links)
→ findet bookNo = 3 (2014) und bookNo = 4 (2019)

b.pubNo = p.pubNo
→ für bookNo = 3 → pubNo = 2 → Publisher: Morgan-Kaufmann
→ für bookNo = 4 → pubNo = 3 → Publisher: Apress

bookNo	title	edition	pubNo	year
1	An Introduction ...	3	1	1981
2	The Relational Model ...	1	1	1990
3	Time and Relational Theory	2	2	2014
4	Database Design ...	2	3	2019

pubNo	publisher	publicity
1	Addison-Wesley	Reading
2	Morgan-Kaufmann	Burlington
3	Apress	New York

Time and Relational Theory	Morgan-Kaufmann
Database Design and Relational ...	Apress

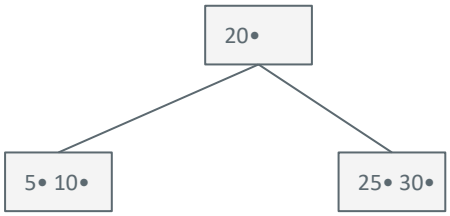
Falls Index auf year: MySQL nutzt einen B^+-Baum, um die bookNo-Werte mit year > 2000 schneller zu finden, als Books-Pages linear zu scannen.

Reale DBMS verwenden nicht den klassischen B-Baum, sondern B^+-Bäume

B^+-Baum

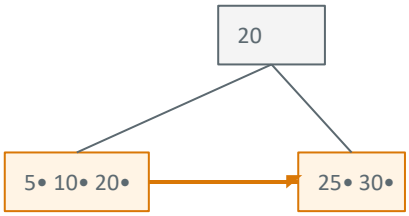
Der „klassische" B-Baum aus der Vorlesung ist ein Lehrbuchkonzept. In der Praxis verwenden praktisch alle DBMS (MySQL / InnoDB, PostgreSQL, Oracle, SQL Server, SQLite) eine Variante: den B^+-Baum.

B-Baum



Datensätze (•) werden in ALLEN Knoten gespeichert – sowohl innen als auch in den Blättern.

B^+-Baum (Praxis)



Datensätze (•) NUR in den Blättern. Innere Knoten sind reine Wegweiser. Blätter sind miteinander verkettet.

Warum der B^+-Baum in der Praxis gewinnt:

1. Mehr Fan-out (mehr Kinder pro Knoten)

Innere Knoten enthalten nur Schlüssel, keine Daten → pro Seite passen viel mehr Verweise → flachere Bäume → weniger I/O.

2. Range-Scans sind billig

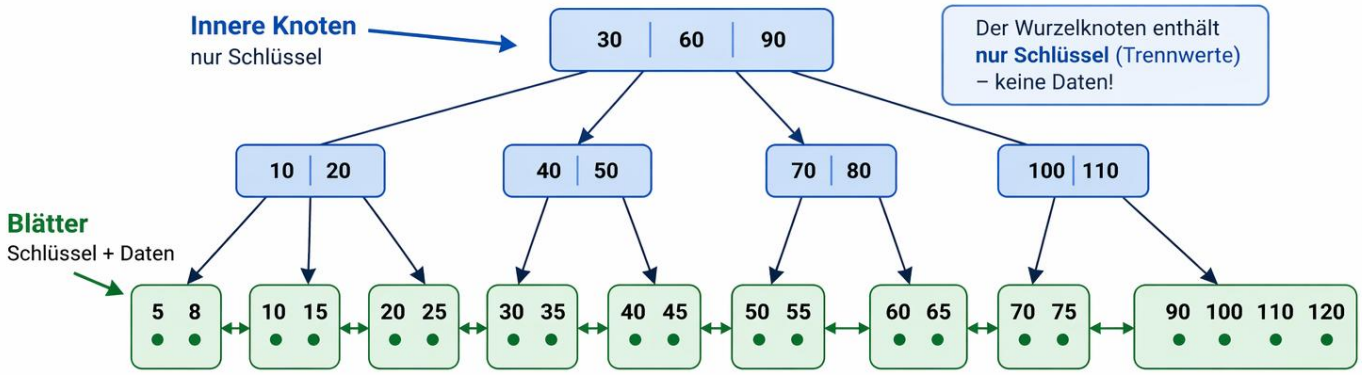
Ein WHERE x BETWEEN a AND b läuft über die Blattkette – nach dem ersten Treffer einfach sequentiell weiterlesen, ohne erneut von der Wurzel zu suchen.

3. Einheitliche Zugriffszeit

Jeder Schlüssel liegt gleich tief (im Blatt). Beim B-Baum hatten Treffer in der Wurzel O(1), Treffer im Blatt O(log n) – beim B^+-Baum immer O(log n), aber vorhersehbar.

B^+-Baum – Komplexeres Beispiel

Innere Knoten enthalten nur Schlüssel. Alle Daten liegen in den Blättern.



Legende

- = Innerer Knoten (nur Schlüssel)
- = Blatt (Schlüssel + Datensätze)
- = Datensatz
- = Verkettung der Blätter für effiziente Bereichsabfragen

Beispiel: Suche nach 65

1. 65 mit 30 | 60 | 90 vergleichen → 65 liegt zwischen 60 und 90 → gehe zum 3. Kind
2. 65 mit 70 | 80 vergleichen → 65 ist kleiner als 70 → gehe zum 1. Kind
3. Im Blatt [60 65] gefunden → Datensatz lesen

Merksatz:
Im B^+-Baum sind **innere Knoten reine Wegweiser**. Die eigentlichen **Daten** liegen **ausschließlich in den Blättern**, die zusätzlich verkettet sind. Dadurch sind Bereichsabfragen besonders effizient.

Übung 1 – B-Baum aufbauen

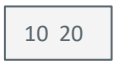
Aufgabe: B-Baum der Ordnung k = 2 (2–4 Schlüssel pro Knoten). Einfügereihenfolge: 10, 20, 5, 30, 25

Lösung – Schritt für Schritt:

Schritt 1: + 10

Erster Schlüssel – wird zur Wurzel.

Schritt 2: + 20



20 > 10, wird rechts angehängt. Wurzel hat noch Platz.

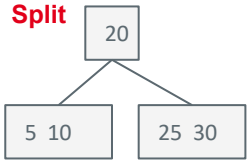
Schritt 3: + 5

5 < 10, sortiert vorne eingefügt.

Schritt 4: + 30

Wurzel ist jetzt voll (max. 4 Schlüssel).

Schritt 5: + 25 → Split



5 Schlüssel → Split: mittlerer Schlüssel (20) wandert hoch, Baumtiefe wächst.

Regel: Sobald ein Knoten die maximale Schlüsselanzahl überschreiten würde, wird er geteilt (Split). Der mittlere Schlüssel steigt eine Ebene auf.

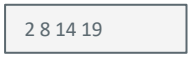
.

Übung 2 – B-Baum mit zwei Splits

Aufgabe: B-Baum der Ordnung k = 2 (2–4 Schlüssel pro Knoten). Füge nacheinander ein: 8, 14, 2, 19, 23, 11, 5, 17, 28, 6, 4. Zeichne den Baum nach jedem Split.

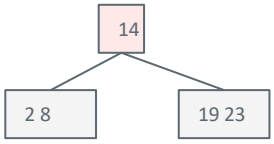
Lösung – sechs Zwischenzustände:

A. Start (nach + 19)



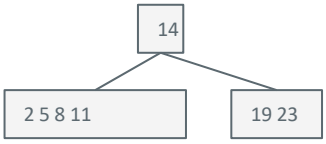
Die ersten vier Schlüssel (8, 14, 2, 19) füllen die Wurzel. Sortiert gespeichert.

B. + 23 → 1. Split



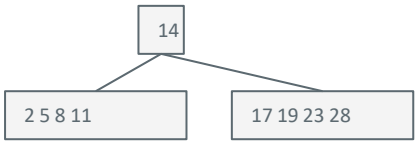
5 Schlüssel → Split. Median 14 wird neue Wurzel.

C. + 11, + 5



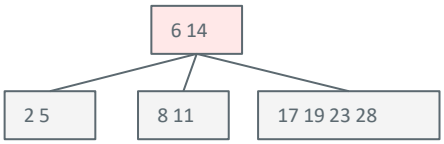
Beide Schlüssel sind < 14 → ins linke Blatt. Es ist jetzt voll.

D. + 17, + 28



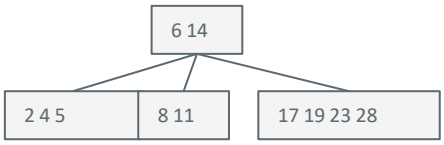
Beide > 14 → ins rechte Blatt. Nun sind beide Blätter voll.

E. + 6 → 2. Split



Linkes Blatt würde 5 Schlüssel haben. Median 6 steigt in die Wurzel auf.

F. + 4 (Endzustand)



4 passt ins linke Blatt. Baum ist 2-stufig mit 3 Blättern.

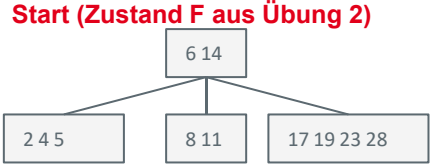
.

Übung 3 – Löschen im B-Baum (Underflow – Knoten hat zu wenig Schlüssel)

Aufgabe: Gegeben ist der Endzustand aus Übung 2 (B-Baum Ordnung k = 2, min. 2 Schlüssel pro Knoten). Lösche nacheinander: 4, 19, 11. Regel bei Underflow (< 2 Schlüssel): Redistribute, falls ein Nachbar > 2 Schlüssel hat – sonst Merge.

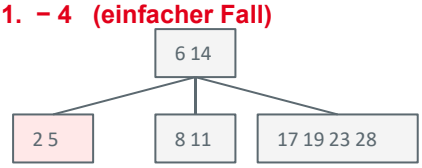
Lösung:

Start (Zustand F aus Übung 2)



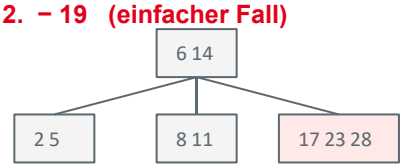
Wurzel [6, 14], drei Blätter, Baum ist 2-stufig.

1. − 4 (einfacher Fall)



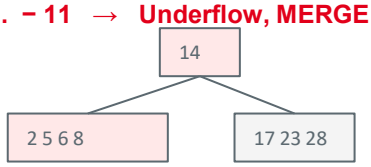
Blatt hat nach dem Löschen noch 2 Schlüssel. Nichts Weiteres zu tun.

2. − 19 (einfacher Fall)



Rechtes Blatt hatte 4 Schlüssel, nach Löschen noch 3. Kein Underflow.

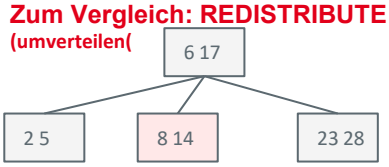
3. − 11 → Underflow, MERGE



Linker Nachbar hat nur 2 Schlüssel → kein Redistribute. MERGE: mittleres + linkes Blatt + Trennschlüssel 6 → [2, 5, 6, 8]. Wurzel schrumpft auf [14].

Zum Vergleich: REDISTRIBUTE

(umverteilen(



Hätte ein Nachbar > 2 Schlüssel: Schlüssel via Wurzel ins unterbesetzte Blatt rotieren. Keine Höhen-Änderung.

Entscheidungsregel

Bei Underflow im Blatt:

1. Hat ein direkter Nachbar mehr als das Minimum → REDISTRIBUTE.

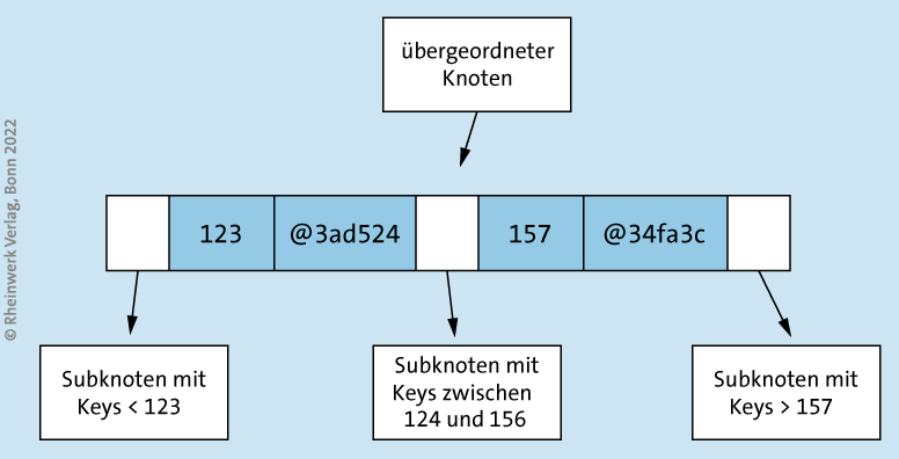
2. Sonst → MERGE mit Nachbar (Trennschlüssel aus Vater wandert ins neue Blatt).

3. Nach MERGE: prüfe Underflow rekursiv im Vater – die Baumhöhe kann sinken.

.

B-Bäume (2) - Beispielbaum der Ordnung 3 (max. 3 Verzweigungen)

B-Bäume der Ordnung 3: Indexeinträge für die IDs 123 und 157. Dazu die hexadezimal codierte Information in welcher Page und wo dort sich der Datensatz befindet. Dazu 3 Links auf untergeordnete Knoten.

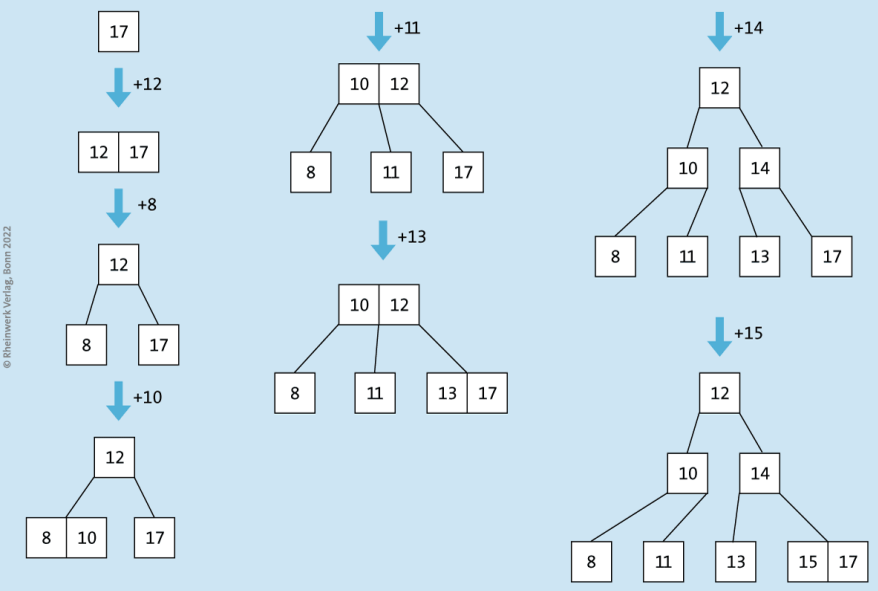


Wurzel mit 2 Schlüssel (123 und 157)

3 Verzweigungen (Zeiger):
→ 1. Links von 123
→ 2. Zwischen 123 und 157
→ 3. Rechts von 157

@3ad524 Hex Pointer für Page-Adresse. Sie zeigen, in welchem Page/Block der nächste untergeordnete Knoten oder Datensatz liegt

Einfügeoperation in einen anfangs leeren B-Tree der Ordnung 3 für die Indexeinträge 17, 12, 8, 10, 11, 13, 14 und 15:



Ein Knoten darf max. 5 Schlüssel haben (2k-1).
Ein Knoten hat max. 6 Kinder (2k).

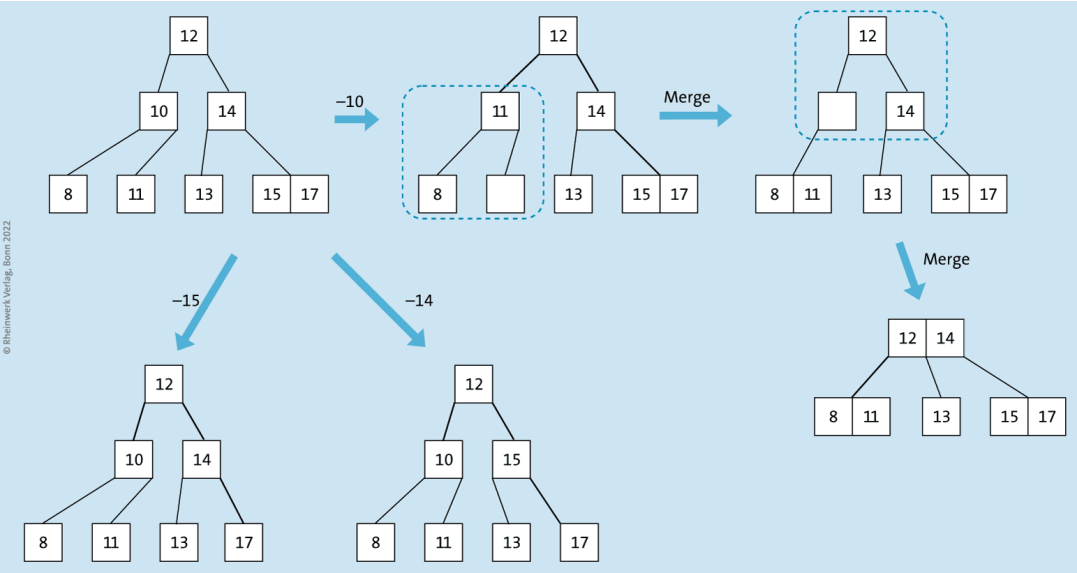
Ein Knoten muss mindestens 2 Schlüssel haben, außer Wurzel)

Die Schlüssel sind sortiert.
Wird ein Knoten zu voll, wird er gesplittet und der mittlere Wert steigt eine Ebene auf.

Split = zentrale Operation zur Balance

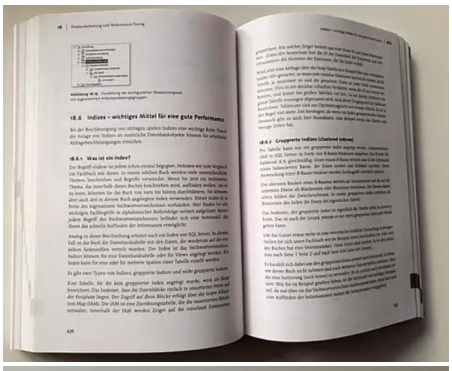
Löschoperation in einem B-Tree der Ordnung 3:

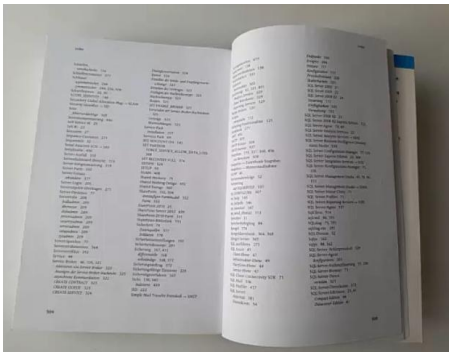
Löschoperationen in B-Bäumen sind komplexer als das Einfügen, da man aufpassen muss, dass die Baumstruktur erhalten bleibt und Mindestanzahl an Schlüsseln pro Knoten beachtet wird.



→ Schlüssel entfernen
→ Unterfüllung erkennen
→ Merge durch Nachbar
→ Ersetzen von innerem Schlüssel

Indexformen (2)





- Clustered Index (“Gruppierter Index”, physische Ordnung der Tabelle)

  Endknoten (Leaves, Blätter) enthalten Indexeinträge samt den dazugehörigen Datensätzen. Im Hinblick auf die indizierte Spalte werden die Datensätze immer geordnet abgespeichert.
  Die Knoten der darüberliegenden Ebenen enthalten ausschließlich Indexeinträge und Querverweise.
  Im SQLServer ist der Primärschlüssel automatisch ein Clusterindex.
  → Das bedeutet, dass die Zeilen physisch auf der Festplatte in der Reihenfolge des Primärschlüssels gespeichert werden.
  Ein Clustered Index ist kein zusätzlicher Index, sondern die eigentliche physische Sortierung der Tabelle.

- Non-clustered Index (“Nicht gruppierter Index”)

  Der Index ist von den eigentlichen Daten einer Tabelle losgelöst: es gibt Pages, die Datensätze enthalten und weitere Pages mit Indexeinträgen, die auf die Datensätze verweisen.
  Die Datensätze sind nicht nach dem Non-Clustered Index geordnet, sondern nach dem Clustered Index. Falls die Datensätze mehr Platz benötigen, müssen sie evtl. in einer neuen Page gespeichert werden.

Vor und Nachteile Clustered/Non Clustered Index

Ein Clustered Index minimiert bei Such- und Lesevorgängen die Anzahl der Pagezugriffe. Besonders z.B. wenn mehrere Datensätze geordnet ausgelesen werden (..BETWEEN… ORDER BY…). Ein Clustered Index eignet sich besonders für Datensätze mit relativ wenig Daten, da man diese alle in einer Page speichern kann. Entsprechend muss man weniger oft Pages zerlegen und neu organisieren.
Allerdings ist die Verwaltung dieses Index aufwändiger. Insofern ist es keine optimale Wahl, wenn man viel Schreib- und Änderungsvorgänge hat.
Pro Tabelle kann es nur einen Clustered Index geben (der Index ist eigentlich die Tabelle selbst in sortierter Form).

MySQL:

```sql
-- Tabelle mit Primärschlüssel (→ Clustered Index)
CREATE TABLE student (
    matrnr INT PRIMARY KEY,  -- Clustered Index (automatisch)
    name VARCHAR(100),
    email VARCHAR(100)
);

-- Normaler Index (→ Non-Clustered Index auf `email`)
CREATE INDEX idx_email ON student(email);
```

In InnoDB ist der Primärschlüssel immer clustered. Falls keiner existiert, wird intern einer erzeugt.

SQL Server:

Welcher Index wann? – Entscheidungshilfe

Welche Indexstruktur für eine Query optimal ist, hängt vom Zugriffsmuster und den Werten der Spalte ab. Die Matrix zeigt, wie gut jeder Indextyp typische Abfragemuster unterstützt.

Zugriffsmuster / Eigenschaft	B^+-Baum	Hash	Bitmap
Gleichheit (WHERE x = ?)	gut	optimal	gut
Range (BETWEEN, <, >)	optimal	unbrauchbar	nur grob
Präfix-Suche (LIKE 'abc%')	optimal	unbrauchbar	unbrauchbar
Sortierung (ORDER BY)	kostenlos	nicht möglich	nicht möglich
Wenige distinct Werte	ineffizient	ineffizient	optimal
Häufige Updates / Inserts	gut	gut	sehr teuer

B^+-Baum

Der universelle Allrounder. Default-Wahl in fast jedem DBMS. Unterstützt Gleichheit, Range, Sortierung, Präfix-LIKE.

Hash

Schneller als B^+-Baum bei reiner Gleichheit, aber nur dort. Kein Range, kein ORDER BY. In der Praxis selten nötig.

Bitmap

Ideal für Data-Warehouse-Queries auf Spalten mit wenigen Werten (z. B. Geschlecht, Status). Teuer bei Updates – OLTP vermeiden.

Indexe in Aktion (MySQL 8)

Tabelle kunden mit 1.000.000 Zeilen (Setup siehe Handout). Jede Demo: erst ohne Index, dann CREATE INDEX, dann erneut ausführen. EXPLAIN ANALYZE zeigt den Plan und die tatsächliche Laufzeit.

① Gleichheit → B+-Baum

```
-- ohne Index
EXPLAIN ANALYZE
SELECT * FROM kunden
WHERE name = 'Kunde_4711';

CREATE INDEX idx_name
  ON kunden (name);
ANALYZE TABLE kunden;
```

✗ **type: ALL ~300 ms**

✓ **type: ref <1 ms**

② Range → B+-Baum

```
EXPLAIN ANALYZE
SELECT COUNT(*) FROM kunden
WHERE registriert BETWEEN
  '2020-01-01'
  AND '2020-12-31';

CREATE INDEX idx_reg
  ON kunden (registriert);
ANALYZE TABLE kunden;
```

✗ **type: ALL ~250 ms**

✓ **type: range ~15 ms**

③ AND-Kombi → Index

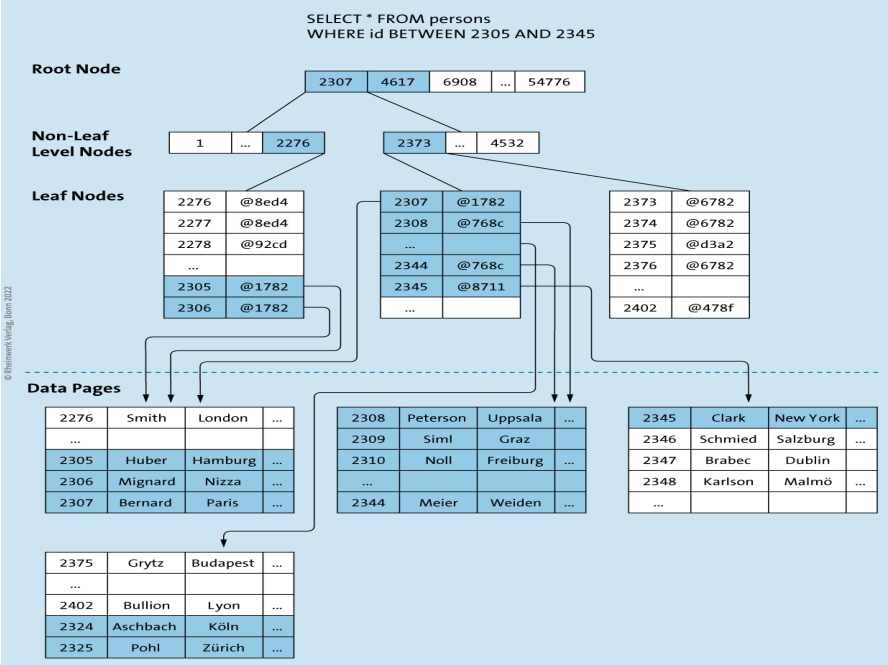
```
CREATE INDEX idx_status
  ON kunden (status);
CREATE INDEX idx_land
  ON kunden (land);
ANALYZE TABLE kunden;

EXPLAIN ANALYZE
SELECT COUNT(*) FROM kunden
WHERE status = 'G'
  AND land   = 'AT';
```

✗ **Ohne: type: ALL ~220 ms**

✓ **Using intersect(...)**

Praxis-Tipps:

- *type: ALL = Full Table Scan, type: ref / range / const = Index wird genutzt.*
- *Nach CREATE INDEX immer ANALYZE TABLE, damit der Optimizer aktuelle Statistiken hat.*
- *MySQL hat keine echten Bitmap-Indexe – bei AND wählt der Optimizer "Index Merge" (intersect / union).*



Schemadarstellung der physikalischen Speicherung mit einem Non-clustered Index: jeder Knoten des B-Baums entspricht einer Page.

→ Der Indexbaum wird durchlaufen, bis die Leaf-Nodes mit id = 2305 bis 2345 gefunden sind

→ Aus den Zeigern (z. B. @1782) springt das DBMS zu den Data Pages.
Die Daten werden nicht sequentiell gelesen (ggf. viele Random I/Os). Mehrere Sprünge auf einzelne Pages.

→ Die Daten werden dann z. B. in persons zurückgeliefert

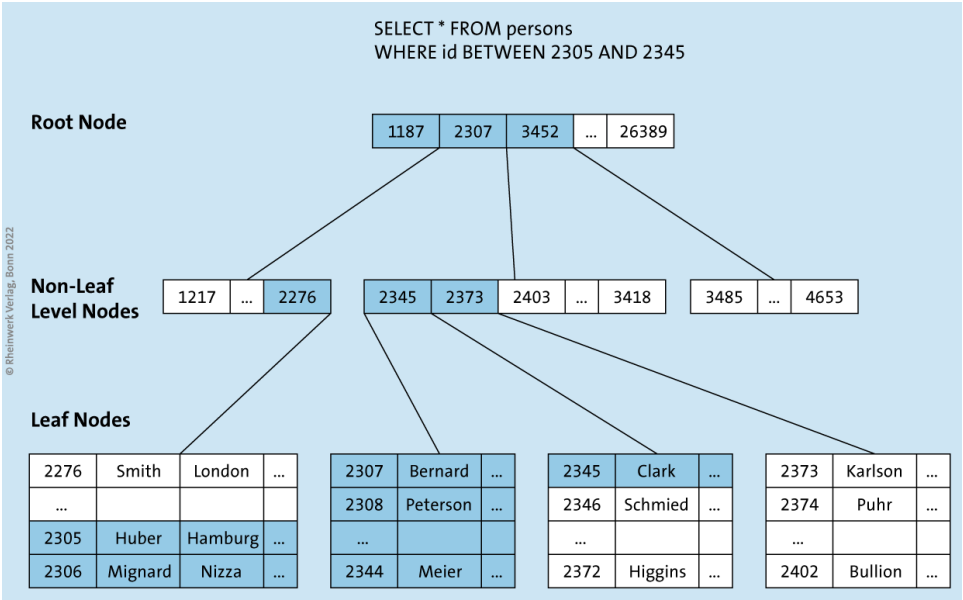


Tabelle mit einem Clustered Index: Jeder Knoten des B-Baums entspricht einer Page.

1. Suche im B-Baum:
Der Baum wird von oben nach unten durchlaufen.
Es wird die untere Grenze 2305 gesucht → Startpunkt im Blatt gefunden.
2. Sequentielles Durchlaufen der Blätter:
Ab dem Blatt mit 2305 wird von links nach rechts gelesen.
Das geht über die verketteten Blattknoten.
Die Abfrage wird so lange fortgesetzt, bis id = 2345 erreicht ist

Art der Spalte	Empfohlener Index
Primärschlüssel	Clustered
Eindeutiger Schlüssel	UNIQUE Index (Non-Clustered)
Suchfelder (email, name, PLZ)	Non-Clustered
Sortier-/Zeitspalten	Clustered oder kombinierter Index

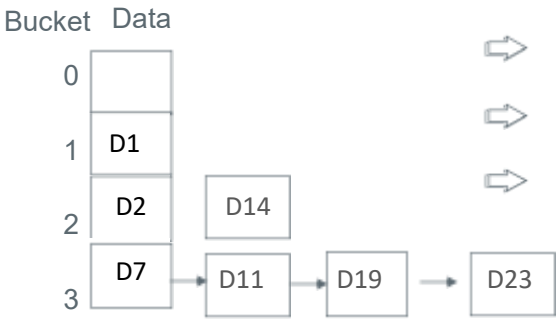
Hash-Verfahren

1. Hashfunktion rechnet Eingabewert deterministisch in Hashwert um (gleicher Eingabewert → gleicher Hashwert
2. In einer Hashtabelle (Datenstruktur, die zur Speicherung und effizienten Suche von Daten verwendet wird) werden Daten abgelegt.
3. Bei neuen Datensätzen wird Hashfunktion auf Schlüssel (Eingabewert) angewendet, um Hashwert zu berechnen. Bei Kollisionen muss Konflikt gelöst werden, um neuen Datensatz korrekt einzufügen.
4. Lesen: Hashfunktion wird auf Schlüssel angewendet, um Speicherort zu bestimmen

→ schneller Zugriff auf Daten.

- Beispiel einer Hashtabelle, die vier Einträge erlaubt und 3 enthält. Die Einträge markieren die Positionen, an denen die Daten basierend auf ihrem Hash-Wert abgelegt werden können. Index 3 enthält eine verkette Liste. Idee: Speicheradresse wird immer (und gleichbleibend) aus dem Schlüssel des Datensatzes berechnet. Die Funktionsvorschrift h(x) bildet den Schlüssel auf einen Behälter ab.

Hash-Table:



(Index 3 enthält eine verkettete Liste)

- Idee: direkte Berechnung der Speicheradresse aus dem Schlüssel eines Datensatzes.
- Gestreute Speicherungsstrukturen. Geeignet für gezielte Anfragen, weniger für Bereichsanfragen.
- Hashfunktion : Abbildung des Schlüssels auf einen Behälter (Bucket) (ggf. eine Speicherseite) $h : S \rightarrow B$ $B = [0..n]$
  In der Regel gilt $|S| >> |B|$!
  - Realisierung : $h(x) = x \bmod n+1 = x \bmod 4$, wobei x der Schlüsselwert ist und (n + 1) die Anzahl der Buckets
  - Nachteil: Bereichsanfragen schwer, Reihenfolge geht verloren

Beispiel für weitere Ideen

- Bitmap-Index

Interessant für Werte mit geringer Anzahl unterschiedlicher Werte (Geschlecht, verheiratet). Das ist sehr effizient, da nur mit Bits gearbeitet wird, nicht mit Strings oder Vergleichen.

Bitmap-Indizes eignen sich besonders bei wenigen Ausprägungen (z. B. Geschlecht, ja/nein) und eher bei leseintensiven Szenarien, nicht bei häufigen Änderungen.

ID	Geschlecht	Alter
2343546	Männlich	18
3267864	Unbekannt	29
3445678	Männlich	2
3435783	Männlich	56
3435788	Weiblich	9
7754233	Männlich	17

ID	m	w	volljährig
2343546	1	0	1
3267864	0	0	1
3445678	1	0	0
3435783	1	0	1
3435788	0	1	0
7754233	1	0	1

```
SELECT * FROM person_bitmap
WHERE m = 1 AND volljährig = 1;
```

```
1 2343546   1 1
2 3435783   1 1
3 7754233   1 1
```

B-Baum als Notebook

Übungen als Jupyter-Notebook auf GitHub:

Notebook in Colab öffnen →

colab.research.google.com/github/karkessler/dhbw-db-2526/blob/app-skeleton-2/Notebooks/btree.ipynb

Vielen Dank für Ihre Aufmerksamkeit!